RoundManager 핵심 동작 분석

개요

RoundManager는 Aptos 컨센서스의 핵심 컴포넌트로, 라운드별 합의 프로토콜을 실행하고 다양한 메시지를 처리하는 역할을 담당합니다. 이 문서는 RoundManager의 메시지 처리 루프, 제안 처리, 투표 처리 과정을 상세히 분석합니다.

3.1 RoundManager 메시지 처리 루프 분석

start() 함수의 이벤트 처리 루프

RoundManager의 `start()` 함수는 컨센서스의 메인 루프로, 다양한 이벤트를 비동기적으로 처리합니다.

함수 시그니처

```
pub async fn start(
mut self,
mut event_rx: aptos_channel::Receiver<
(Author, Discriminant<VerifiedEvent>),
(Author, VerifiedEvent),
>,
mut buffered_proposal_rx: aptos_channel::Receiver<Author, VerifiedEvent>,
mut opt_proposal_loopback_rx: aptos_channels::UnboundedReceiver<OptBlockData>,
close_rx: oneshot::Receiver<oneshot::Sender<()>>,
```

```
)
```

메인 이벤트 루프 구조

메인 루프는 `tokio::select!` 매크로를 사용하여 여러 이벤트 소스를 동시에 모니터링합니다:

```
loop {
tokio::select! {
biased;
// 1. 종료 요청 처리
close_req = close_rx.select_next_some() => {
if let Ok(ack_sender) = close_req {
ack_sender.send(()).expect("[RoundManager] Fail to ack shutdown");
}
break;
}
// 2. 최적화 제안 루프백 처리
opt_proposal = opt_proposal_loopback_rx.select_next_some() => {
// 최적화 제안 처리 로직
}
// 3. 버퍼된 제안 처리
proposal = buffered_proposal_rx.select_next_some() => {
// 제안 메시지 배치 처리
}
```

```
// 4. 페이로드 가져오기 완료 처리

Some((result, block, start_time)) = self.futures.next() => {

// 비동기 페이로드 가져오기 결과 처리

}

// 5. 일반 네트워크 이벤트 처리

(peer_id, event) = event_rx.select_next_some() => {

// 투표, 타임아웃, 동기화 메시지 처리

}

}

}
```

이벤트 처리 우선순위

`biased` 키워드를 사용하여 이벤트 처리에 우선순위를 부여합니다:

1. **종료 요청**: 가장 높은 우선순위로 즉시 처리
2. **최적화 제안**: 성능 최적화를 위한 특별 처리
3. **버퍼된 제안**: 제안 메시지의 배치 처리
4. **페이로드 가져오기**: 비동기 작업 완료 처리
5. **일반 네트워크 이벤트**: 투표, 타임아웃 등 일반적인 컨센서스 메시지

UnverifiedEvent에서 VerifiedEvent로의 변환 과정

UnverifiedEvent 구조

네트워크에서 수신된 메시지는 먼저 `UnverifiedEvent` 로 분류됩니다:

```
#[derive(Debug, Serialize, Clone)]
```

```
pub enum UnverifiedEvent {
ProposalMsg(Box<ProposalMsg>),
VoteMsg(Box<VoteMsg>),
RoundTimeoutMsg(Box<RoundTimeoutMsg>),
OrderVoteMsg(Box<OrderVoteMsg>),
SyncInfo(Box<SyncInfo>),
BatchMsg(Box<BatchMsg>),
SignedBatchInfo(Box<SignedBatchInfoMsg>),
ProofOfStoreMsg(Box<ProofOfStoreMsg>),
OptProposalMsg(Box<OptProposalMsg>),
}
```

검증 과정 (verify 함수)

`UnverifiedEvent::verify()` 함수는 다음 단계를 통해 메시지를 검증합니다:

1. **서명 검증**: 메시지 발신자의 디지털 서명 확인
2. **에포크 검증**: 현재 에포크와 메시지 에포크 일치 확인
3. **구조적 검증**: 메시지 형식과 필수 필드 검증
4. **권한 검증**: 발신자가 해당 메시지를 보낼 권한이 있는지 확인

```
pub fn verify(
self,
peer_id: PeerId,
validator: &ValidatorVerifier,
```

```
proof_cache: &ProofCache,
quorum_store_enabled: bool,
self_message: bool,
max_num_batches: usize,
max_batch_expiry_gap_usecs: u64,
) -> Result<VerifiedEvent, VerifyError>
```

VerifiedEvent 구조

검증이 완료된 메시지는 `VerifiedEvent` 로 변환됩니다:

```
#[derive(Debug)]
pub enum VerifiedEvent {
// 네트워크 메시지
ProposalMsg(Box<ProposalMsg>),
VerifiedProposalMsg(Box<Block>),
VoteMsg(Box<VoteMsg>),
RoundTimeoutMsg(Box<RoundTimeoutMsg>),
OrderVoteMsg(Box<OrderVoteMsg>),
UnverifiedSyncInfo(Box<SyncInfo>), // 지연 검증
BatchMsg(Box<BatchMsg>),
SignedBatchInfo(Box<SignedBatchInfoMsg>),
ProofOfStoreMsg(Box<ProofOfStoreMsg>),
// 로컬 메시지
LocalTimeout(Round),
```

```
Shutdown(TokioOneshot::Sender<()>),
OptProposalMsg(Box<OptProposalMsg>),
}
```

검증 최적화

- **자체 메시지 스킵**: `self_message` 가 true인 경우 검증 생략
- **지연 검증**: SyncInfo는 사용 시점에 검증 (`UnverifiedSyncInfo`)
- **성능 모니터링**: 각 메시지 타입별 검증 시간 측정

```
counters::VERIFY_MSG
.with_label_values(&["proposal"])
.observe(start_time.elapsed().as_secs_f64());
```

에러 처리 및 복구 메커니즘

에러 분류 및 처리

각 이벤트 처리 결과는 다음과 같이 처리됩니다:

```
match result {
Ok(_) => trace!(RoundStateLogSchema::new(round_state)),
Err(e) => {
counters::ERROR_COUNT.inc();
warn!(kind = error_kind(&e), RoundStateLogSchema::new(round_state), "Error: {:#}
}
```

```
}
```

모니터링 및 메트릭

각 처리 단계는 `monitor!` 매크로로 감싸져 성능 메트릭을 수집합니다:

```
let result = monitor!("process_vote", self.process_vote_msg(*vote_msg).await);
```

백프레셔 및 성능 최적화

제안 메시지 배치 처리

버퍼된 제안들은 라운드 순서로 정렬되어 배치 처리됩니다:

```
proposals.sort_by_key(get_round);
// 첫 번째 제안이 다음 라운드가 아니면 마지막 제안만 처리
if self.round_state.current_round() + 1 < get_round(&proposals[0]) {
proposals = vec![proposals.pop().unwrap()];
}
```

비동기 페이로드 처리

페이로드 가져오기는 별도의 Future로 처리되어 메인 루프를 블록하지 않습니다:

```
Some((result, block, start_time)) = self.futures.next() => {
let elapsed = start_time.elapsed().as_secs_f64();
match result {
Ok(()) => {
```

```
counters::CONSENSUS_PROPOSAL_PAYLOAD_FETCH_DURATION

.with_label_values(&["success"])

.observe(elapsed);

},

Err(err) => {

counters::CONSENSUS_PROPOSAL_PAYLOAD_FETCH_DURATION

.with_label_values(&["error"])

.observe(elapsed);

}

}

}
```

이러한 메시지 처리 루프 설계를 통해 RoundManager는 높은 처리량과 낮은 지연시간을 달성하면서도 안정적인 컨센서스 동작을 보장합니다.